Reporte: 2p ejerccio 9 cola

En este programa implementamos una Estructura de Datos de tipo Cola (Queue) bajo el principio FIFO (First In, First Out - El primero en entrar es el primero en salir) utilizando la clase estándar <queue> proporcionada por la librería estándar de C++ (STL). Todo esto se estructuró aplicando Programación Orientada a Objetos mediante la clase ColaDinamicaIDE.

Código Completo:

```
#include <iostream>
#include <queue> // Librería estándar de C++ para la cola (queue)
using namespace std;

class ColaDinamicaIDE {
private:
    queue<int> cola; // Cola dinámica proporcionada por la librería estándar (STL)

public:
    // Método para insertar un elemento (Enqueue)
    void encolar(int valor) {
        cola.push(valor);
        cout << "Elemento " << valor << " agregado a la cola.\n";
    }

    // Método para eliminar y extraer el elemento del frente (Dequeue)
    void desencolar() {
        if (estaVacia()) {
            cout << "Error: La cola esta vacia. No se puede desencolar.\n";
            return;
        }
        cout << "Elemento " << cola.front() << " eliminado de la cola.\n";
        cola.pop();
    }

    // Método para ver el elemento al frente sin eliminarlo (Front)
    void verFrente() const {
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }
        cout << "Elemento al frente de la cola: " << cola.front() << "\n";
    }

    // Verificar si la cola está vacía
    bool estaVacia() const {
        return cola.empty();
    }

    // Obtener el tamaño actual de la cola
    void mostrarTamano() const {
        cout << "Tamaño actual de la cola: " << cola.size() << "\n";
    }

    // Mostrar todos los elementos (creando una copia para no vaciar la original)
    void mostrarCola() const {
```

```
        if (estaVacia()) {
            cout << "La cola esta vacia.\n";
            return;
        }
        queue<int> temp = cola; // Copia de la cola para iterar
        cout << "Cola (Frente -> Final): [ ";
        while (!temp.empty()) {
            cout << temp.front() << " ";
            temp.pop();
        }
        cout << "]\n";
    }
};

int main() {
    ColaDinamicaIDE gestorCola;
    int opcion, valor;

    do {
        cout << "\n======================================\n";
        cout << "   COLA DINÁMICA (STL queue) - C++    \n";
        cout << "======================================\n";
        cout << "1. Encolar (push)\n";
        cout << "2. Desencolar (pop)\n";
        cout << "3. Ver frente (front)\n";
        cout << "4. Ver tamaño (size)\n";
        cout << "5. Mostrar cola completa\n";
        cout << "6. Salir\n";
        cout << "Selecciona una opción: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << "Ingresa el valor a encolar: ";
                cin >> valor;
                gestorCola.encolar(valor);
                break;
            case 2:
                gestorCola.desencolar();
                break;
            case 3:
                gestorCola.verFrente();
                break;
            case 4:
                gestorCola.mostrarTamano();
                break;
            case 5:
                gestorCola.mostrarCola();
                break;
            case 6:
                cout << "Saliendo del programa...\n";
                break;
            default:
                cout << "Opción no válida. Inténtalo de nuevo.\n";
        }
    } while (opcion != 6);

    return 0;
```

```
}
```

Explicación del Código:

1. Uso de la Librería Estándar (queue) y Encapsulamiento (private:)
Se incluye la cabecera <queue> que provee la estructura de datos optimizada del IDE. La instancia cola se declara como privada dentro de la clase ColaDinamicaIDE, asegurando que solo se pueda manipular a través de los métodos públicos definidos.

2. Inserción de Elementos (Método encolar / push)
Utiliza la función nativa push() para agregar nuevos elementos al final de la cola de manera dinámica, sin necesidad de gestionar capacidades fijas ni desbordamientos manuales.

3. Extracción y Consulta (Métodos desencolar / pop y verFrente / front)
Verifica primero que la cola no esté vacía con empty(). Emplea front() para consultar el elemento que se encuentra al frente y pop() para retirarlo, cumpliendo estrictamente con el comportamiento FIFO.

4. Visualización y Control de Tamaño (mostrarCola y mostrarTamano)
Permite conocer el número exacto de elementos mediante size(). Para mostrar la cola completa sin perder los datos originales, se genera una copia temporal (queue<int> temp = cola) sobre la cual se iteran y extraen los elementos.

5. Menú Interactivo en la Función Principal (main)
Instancia un objeto de tipo ColaDinamicaIDE llamado gestorCola y gestiona un menú interactivo mediante un bucle do-while y un bloque switch-case para realizar operaciones fluidas.